

Spam Detection:

A Machine Learning Approach

Introduction

Spam emails continue to be one of the most common and persistent problems in digital communication. They can clutter inboxes, waste time, and in many cases expose users to security risks such as phishing attempts, malware, and financial scams. As email remains a primary method of communication for both personal and professional use, having reliable tools that can automatically detect and filter out unwanted messages is essential.

Data Mining Task

The project is formulated as a binary classification problem. The goal is to learn a model that maps each email's feature vector to a predicted class label: 0. (non-spam) or 1 (spam).. For this project, spam detection was chosen because it represents a real-world classification problem that is both practical and meaningful. The dataset is well-known, labeled, and suitable for applying and comparing different machine learning algorithms. Building an effective spam classifier can help improve email security, reduce user exposure to harmful content, and streamline overall inbox management. By evaluating several models and selecting the most accurate one, this project demonstrates how data-driven methods can address everyday challenges in a direct and impactful way.

Data source

- **Kind of data.**
 - We use the UCI Spambase dataset, which consists of numeric features extracted from email text (word and character frequencies) and a binary label indicating whether an email is spam (1) or legitimate (0).
- **Dataset information.**
 - The dataset contains 4,601 instances and 58 attributes in total: 57 numeric input features and 1 binary target attribute. There are no missing values. The label distribution includes 2,788 non-spam emails and 1,813 spam emails
- **Data processing.**
 - All 57 numeric attributes were retained. The data were split into features X and label y, where y is the final column. Features were normalized to the [0, 1] range using Min-Max scaling. The dataset was split into training and testing sets using

an 80/20 stratified split with random_state = 42 to ensure reproducibility and preserve the original spam/non-spam ratio.

- **Attributes used and not used.**
 - All original numeric features provided by Spambase were used; no attributes were dropped. Because the dataset is already curated and fully numeric, and missing-value issues were observed, there was no need to exclude features. Feature importance analysis with the Random Forest confirmed that many attributes contribute meaningful signals to the classification task.

Preprocessing used & Implemented Methodologies

Four different models were implemented and evaluated

- **Naive Bayes (GaussianNB):** Naive Bayes is a simple probabilistic classifier that assumes the features are independent given the class label. It was used because it is fast, handles high-dimensional data well, and provides a solid baseline for comparison.
- **Logistic Regression (with scaling):** Logistic Regression is a linear model that predicts the probability of a sample belonging to a class using a logistic function. It was chosen because it is dependable, easy to interpret, and performs well once the features are scaled.
- **Decision Tree Classifier:** A Decision Tree is a model that splits the data into branches based on feature values to reach a classification decision. It was included because it can capture non-linear patterns and gives a clear visual representation of how the model makes decisions.
- **Random Forest Classifier:** Random Forest is an ensemble method that builds many decision trees and averages their predictions to improve accuracy. It was used because it reduces overfitting, increases stability, and typically performs extremely well on structured datasets like this one.

Results & Outcome of Code

The following test accuracies were obtained

Model	Test Accuracy
Naive Bayes	0.8240
Logistic Regression	0.9283

Decision Tree	0.8921
Random Forest	0.9558

Best Model:

Among all models, the Random Forest achieved the highest test accuracy of 0.9558 and was therefore selected as the final model. The confusion matrix shows the distribution of true positives, true negatives, false positives, and false negatives. This model correctly classifies the large majority of both spam and non-spam messages, with relatively few misclassifications

- Overall accuracy: 0.96
- For class 0 (non-spam): precision 0.95, recall 0.97, F1-score 0.96 (support = 837)
- For class 1 (spam): precision 0.96, recall 0.93, F1-score 0.94 (support = 544)
- Macro and weighted averages of precision/recall/F1 are all approximately 0.95–0.96

The ROC (Receiver Operating Characteristic) curve, plotted in the workbook for the Random Forest using predicted class probabilities, yields an AUC (Area Under the Curve) of approximately 0.986. This indicates excellent separability between spam and non-spam across all possible classification thresholds.

Discussion on Results

Cross-Validation and Stability

- To assess stability, a cloned instance of the best model was evaluated using 5-fold cross-validation on the entire dataset. The resulting fold accuracies are tightly clustered around the mid-90% range, and the mean cross-validated accuracy is close to the single train/test split accuracy. This confirms that the Random Forest model generalizes well and is not simply overfitting a particular split.

Hyperparameter Tuning and Feature Importance

- For the Random Forest, a grid search over the number of trees, maximum depth, and minimum samples required for splitting was conducted using GridSearchCV with 3-fold cross-validation on the training set. The best-performing hyperparameter combination improved or matched the baseline Random Forest accuracy on the test set, confirming that the chosen configuration is near-optimal for this dataset.
- Feature importance values from the fitted Random Forest indicate which input attributes are most influential in distinguishing spam from non-spam emails. Several high-importance features correspond to word and character frequencies commonly

associated with spammy content, which provides an interpretable justification for the model's decisions.

Final Result

- Overall, the Random Forest classifier provides the best trade-off between accuracy and robustness among the models tested. Its high precision and recall for both classes, together with the strong ROC AUC, suggest that it would be effective as a backbone for an automated spam-filtering system. Naive Bayes offers a simpler and faster baseline but with noticeably lower accuracy, while Logistic Regression and Decision Trees perform reasonably well but are outperformed by the ensemble method

Conclusion and Outcome Implementations

- This project demonstrates an end-to-end supervised learning workflow for spam-email detection: data loading, preprocessing, model comparison, selection of a best model, detailed evaluation, and artifact generation. Future work could include experimenting with additional classifiers as well as applying it to exported .csv from an individual's email to predict spam messages with an automated prompt or to predict helpful messages and alert oneself.

References

1. UCI Machine Learning Repository: Spambase Data Set.
2. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, 2011.